

Many Bodies, One GIF Pack

ESP32 firmware + WebAssembly preview

หนึ่งแพ็ค หลายร่าง หนึ่งสัญญา



digital desk-pet workshop

browser preview firmware exported proof

Many Bodies, One GIF Pack

จาก desk-pet character pack เดียว สู่ firmware บน ESP32 และ preview ด้วย WebAssembly

Atom Oracle

2026-06-17

คำเปิด

เล่มนี้ไม่ได้เริ่มจากคำถามว่า “เขียน firmware ยังไง” แต่เริ่มจากคำถามที่สำคัญกว่า: เราอยู่ผิดระบบหรือเปล่า

ช่วงแรกของงานมีความเข้าใจผิดว่า submission ต้องไปทาง ESPHome หรือ wasm3 serial printer แต่หลักฐานใน source ชี้อีกทางหนึ่ง firmware ที่เกี่ยวข้องคือ jc3248-pet-idf และ web preview ที่เกี่ยวข้องคือ gif-wasm สิ่งที่ต้องการไม่ใช่ YAML ใหม่ แต่เป็น character pack ที่ถูก decode ได้จาก GIF ชุดเดียวกัน

แก่นของงานจึงสั้นมาก:

```
LittleFS /characters/<pack>/*.gif
-> AnimatedGIF decoder
-> LovyanGFX Sprite
-> display
```

```
web/gifs/<pack>/*.gif
-> gif-wasm
-> Canvas preview
```

ถ้า pack เดียวกันรันได้ทั้งสองร่าง คนตรวจจะเห็นภาพเดียวกับ firmware path มากขึ้น ถ้า preview เป็นภาพตัวเองแต่ device ใช้อีก asset หนึ่ง หลักฐานจะขาดใจกลางทันที

เล่มนี้บันทึกวิธีสร้าง digital เป็น character pack ใหม่แบบ reproducible, วิธี validate ด้วย WASM decoder, วิธีสร้าง LittleFS proof และวิธีรายงาน compile boundary อย่างตรงไปตรงมา

ขอบเขต

เล่มนี้ครอบคลุม:

- การอ่าน source เพื่อหา lane ที่ถูก
- รูปแบบ character pack ของ desk-pet
- การสร้าง GIF state set ขนาด 96x100
- การเพิ่ม pack ใน firmware data และ web preview
- การ validate ด้วย identify, Node + gifdec.wasm, LittleFS image และ browser screenshot
- การส่ง proof เป็น attachment ที่ทีมเปิดดูได้จริง

เล่มนี้ยังไม่ claim ว่า firmware binary compile ผ่านในเครื่องนี้ เพราะ environment ยังไม่มี ~/esp/esp-idf และไม่มี python3.13 ตามที่ source pin ไว้

บทที่ 1: ปัญหาที่ไม่ใช่ ESPHome

ความผิดพลาดที่แพงที่สุดใน workshop แบบนี้ไม่ใช่ syntax error แต่คือการทำงานใน lane ที่ผิด

ตอนแรกโจทย์ฟังเหมือน firmware ทั่วไป: มี ESP32, มี web flasher, มี WASM, มี LVGL, และมีคำว่า ESPHome โผล่มาในบทสนทนา ถ้ารับตอบจาก keyword ก็จะไปสร้าง esphome/ folder, YAML, binary อีกแบบหนึ่ง แล้วอาจ compile ผ่านด้วยซ้ำ แต่ pass แบบนั้นไม่ได้แปลว่าตรงกับระบบที่พื้นหลังมา

หลักฐานใน source บอกชัดกว่า keyword:

```
lab/jc3248-pet-idf/
lab/gif-wasm/
```

ชื่อ character ที่ผู้ใช้ให้หาไม่ได้เป็น component ของ ESPHome แต่เป็น pack name ใน desk-pet:

bufo

พอเห็นชื่อนี้พร้อม folder asset แล้วภาพจริงของระบบก็เปลี่ยนทันที งานไม่ได้อยู่ที่การสร้าง firmware framework ใหม่ แต่อยู่ที่การทำ character pack ให้ตรง contract เดิม

จุดที่ต้องจำคือ web flasher กับ web preview ไม่ใช่หลักฐานเดียวกัน web flasher อาจ install binary ได้ แต่ preview ต้องแสดง asset เดียวกับที่ firmware เล่น ถ้าฝั่งเว็บแสดงภาพหนึ่ง ส่วน firmware เล่นอีกภาพหนึ่ง คนตรวจจะเชื่อไม่ได้ว่าเราส่งของถูก

ดังนั้นคำสั่งแรกของงานนี้ไม่ใช่ build แต่เป็น search:

```
rg -n -I "jc3248-pet-idf|gif-wasm|AnimatedGIF|LovyanGFX|manifest.json" "$WORK"
find "$WORK" -type f -iname '*.gif' | sort
```

การ search นี้ทำให้เห็นว่า source มี two-body architecture อยู่แล้ว: device body และ browser body ทั้งคู่กิน GIF pack เป็น input เดียวกัน คนที่ส่งงานจึงต้องรักษา contract ตรงนี้ ไม่ใช่สร้างระบบขนานอีกชุด

บทเรียนของบทนี้คือ “อ่าน source ก่อนสร้างคำตอบ” เพราะ source เป็นตัวตัดสินว่าโจทย์จริงอยู่ที่ไหน

บทที่ 2: หา desk-pet characters ใน source

การหา bufo, cat, cat-orange, cat-pet, และ clawd เป็นการหา contract ไม่ใช่หา mascot

เมื่อเปิด source แล้วรูปแบบสำคัญคือ folder ต่อ pack:

```
data/characters/<pack>/
web/gifs/<pack>/
```

ใน firmware side, pack อยู่ใต้ lab/jc3248-pet-idf/data/characters/ และมี manifest.json เป็นตัวประกาศ state map ส่วนใน web side, pack อยู่ใต้ lab/gif-wasm/web/gifs/ และถูกเลือกผ่าน PACKS ใน index.html

ชื่อ state เป็นภาษากลางของระบบ:

```
idle
busy
attention
celebrate
dizzy
heart
sleep
```

บาง pack มีชื่อพิเศษ เช่น clawd-idle หรือ clawd-typing แต่ pattern รวมยังเหมือนเดิม คือ browser ต้องหา GIF ได้และ decoder ต้องอ่าน frame ได้ ส่วน device ต้อง pack ลง LittleFS แล้ว AnimatedGIF อ่านได้

สิ่งที่น่าสนใจคือ contract นี้เล็กมาก ไม่มี database ไม่มี API ไม่มี build graph ซับซ้อน มีแค่ folder, JSON, GIF และชื่อ state แต่เพราะมันเล็ก มันจึงต้องตรง ถ้า state หายหนึ่งตัว preview อาจเลือกไม่ได้ ถ้า GIF dimension ผิด device layout อาจเพี้ยน ถ้า path ผิด web ไม่ตรงกับ path ผิด firmware proof ก็หลุดจากกัน

วิธีคิดที่เข้ากับ digimal จึงคือ copy shape ไม่ใช่ copy content:

```
characters/digimal/
  manifest.json
  idle.gif
  busy.gif
  attention.gif
  celebrate.gif
  dizzy.gif
```

heart.gif
sleep.gif

พอ shape ตรง ระบบเดิมก็ทำงานแทนเรา ที่เหลือคือทำให้ asset มีคุณภาพพอและพิสูจน์ว่า decode ได้จริง

บทที่ 3: Many bodies, one GIF pack

หัวใจของ desk-pet pipeline คือประโยคเดียว:

many bodies, one GIF pack

ร่างแรกคือ ESP32 firmware ไฟล์ GIF อยู่ใน LittleFS path แล้ว native AnimatedGIF decoder อ่าน frame ออกมา ส่งต่อเข้า sprite และ push ไปที่จอ

ร่างที่สองคือ browser preview ไฟล์ GIF อยู่ใน web/gifs/ แล้ว gif-wasm decode ด้วย WebAssembly จากนั้น draw ลง Canvas ให้คนตรวจเห็นก่อน flash

ถ้าสองร่างนี้ใช้ GIF คนละชุด เราจะได้ demo ที่ดูดีแต่ไม่พิสูจน์ firmware ถ้าสองร่างนี้ใช้ GIF ชุดเดียวกัน pre-view จะกลายเป็นหลักฐานที่มีน้ำหนักกว่า screenshot ธรรมดา เพราะมัน decode ผ่าน engine ที่ออกแบบมาเพื่อตรวจทางเดียวกับ runtime จริง

ในงานนี้ digimal ถูก copy ไปสองที่:

jc3248-pet-idf/data/characters/digimal/
gif-wasm/web/gifs/digimal/

นี่ไม่ใช่ duplication แบบไร้เหตุผล แต่เป็นการวาง asset เดียวกันในสอง packaging context firmware ต้องการ LittleFS tree ส่วนเว็บต้องการ static web path

แนวทางนี้ยังทำให้ review ง่ายขึ้น เพื่อนในที่มไม่ต้ออ่าน C++ ทั้งหมดก่อนเห็นผล เขาเปิด preview แล้วเห็น animation ได้ จากนั้นค่อยอ่าน proof ว่า GIF เดียวกันอยู่ใน firmware data และ LittleFS image ได้จริง

ระบบที่ดีใน workshop ไม่ใช่ระบบที่ซับซ้อนที่สุด แต่คือระบบที่พิสูจน์ได้จากหลายมุมโดยใช้ input เดียวกัน

บทที่ 4: Character pack contract

digimal pack มี contract หลัก 4 ข้อ:

1. ทุก GIF เป็น GIF89a
2. ทุก GIF ขนาด 96x100
3. ทุก state ที่ manifest อ้างถึงมีไฟล์จริง
4. ไฟล์ชุดเดียวกันอยู่ทั้ง firmware data และ web preview

manifest.json ของ pack เป็นแบบนี้:

```
{  
  "name": "digimal",  
  "colors": {  
    "body": "#4ADE80",  
    "bg": "#060A18",  
    "text": "#FFFFFF",  
    "textDim": "#93A4B8",  
    "ink": "#020617"  
  },  
  "states": {  
    "sleep": "sleep.gif",
```

```

    "idle": ["idle.gif"],
    "busy": "busy.gif",
    "attention": "attention.gif",
    "celebrate": "celebrate.gif",
    "dizzy": "dizzy.gif",
    "heart": "heart.gif"
  }
}

```

จุดที่ควรสังเกตคือ idle เป็น array ได้ ส่วน state อื่นเป็น string ได้ นี่สะท้อน shape ของ pack เดิม ไม่ใช่ schema ที่คิดขึ้นใหม่

ในระบบแบบนี้ manifest ไม่ควรฉลาดเกินไป หน้าที่ของมันคือประกาศชื่อ state และไฟล์ที่ state นั้นใช้ ส่วน animation timing อยู่ใน GIF เอง ถ้าเอา logic จำนวนมากไปใส่ manifest เราจะต้องแก้ทั้ง firmware และ web parser เพิ่มโดยไม่จำเป็น

contract เล็กแบบนี้เหมาะกับ workshop เพราะทุกคนตรวจได้ด้วย command พื้นฐาน:

```

file data/characters/digimal/*.gif
identify data/characters/digimal/*.gif

```

ถ้าไฟล์ไม่ครบหรือ dimension ผิด จะเห็นทันทีโดยไม่ต้อง flash board

บทที่ 5: Generate GIF sprite 96x100

สำหรับ workshop asset ที่ดีควร reproduce ได้ ไม่ใช่เป็นไฟล์ล็อกที่ไม่มีใครมีที่มา

digimal ถูกสร้างเป็น original sprite ด้วย Python + Pillow ขนาด 96x100 ทุก state ใช้ base drawing เดียวกัน แล้วเปลี่ยน timing หรือ expression ตาม state วิธีนี้ทำให้ asset มีเอกลักษณ์ ไม่แตะ IP ของ Digimon และยังแก้ไขได้ถ้าขนาดหรือสีต้องเปลี่ยน

state set ที่สร้างคือ:

idle	8 frames
busy	6 frames
attention	5 frames
celebrate	8 frames
dizzy	6 frames
heart	5 frames
sleep	6 frames

การทำ sprite ด้วย code มีข้อดีสามอย่าง

ข้อแรกคือ versioning ชัด ถ้าจะเปลี่ยนสีหรือขนาด แก้ script แล้ว generate ใหม่ ไม่ต้องเปิด editor แล้วจำไม่ได้ว่าไฟล์ไหนมาจากไหน

ข้อสองคือ proof ชัด เราสามารถบอกได้ว่าไฟล์ทั้งหมดควรเป็น 96x100 และใช้ palette แบบ GIF ได้

ข้อสามคือลดความเสี่ยงเรื่องลิขสิทธิ์ เพราะ character เป็น original shape ที่สร้างเอง ไม่ใช้การ import sprite จากแฟรนไชส์อื่น

หลัง generate แล้วต้อง copy ไปทั้ง firmware และ web:

```

characters/digimal/*.gif
web/gifs/digimal/*.gif

```

ถ้าขั้นนี้ทำด้วย script เดียว ความเสี่ยงที่สองฝั่งไม่ตรงกันจะลดลงมาก

บทที่ 6: เพิ่ม pack ใน firmware builder

ฝั่ง firmware ไม่ต้องรู้จัก digimal ผ่าน C++ ก่อนก็ได้ สิ่งแรกที่ต้องทำคือให้ pack builder เห็น folder นี้

จุดแก้ที่แคบที่สุดคือ default pack list ใน:

lab/jc3248-pet-idf/tools/build-packs.sh

รูปแบบที่ต้องการคือให้ command ที่ไม่ได้ระบุ pack รวม digimal ด้วย:

```
PACKS=("$@"); [ ${#PACKS[@]} -eq 0 ] && PACKS=(bufo cat cat-orange cat-pet clawd digimal)
```

นี่เป็นการ integrate ที่ conservative เพราะไม่ได้แตะ decoder, display driver, event loop หรือ UI logic ใด ๆ ถ้า pack contract ตรง ระบบเดิมควรจัดการต่อได้

ใน firmware pipeline, asset ไม่ได้อยู่ใน app binary โดยตรง แต่อยู่ใน LittleFS image ดังนั้นการ validate asset packaging มีความหมายของมันเอง แม้ app compile จะยังติด environment อยู่ก็ตาม

คำสั่ง proof ที่ใช้คือสร้าง LittleFS image จาก data/:

```
uvx --from littlefs-python python - <<'PY'
from littlefs import LittleFS
from pathlib import Path
root = Path('data')
out = Path('/tmp/digimal-storage-proof.bin')
fs = LittleFS(block_size=4096, block_count=768)
for p in sorted(root.rglob('*')):
    if p.is_file():
        rel = '/' + str(p.relative_to(root))
        fs.makedirs(str(Path(rel).parent), exist_ok=True)
        with fs.open(rel, 'wb') as f:
            f.write(p.read_bytes())
out.write_bytes(fs.context.buffer)
PY
```

proof นี้ตอบคำถามว่า “ไฟล์ pack เข้า storage image ได้ไหม” ไม่ได้ตอบว่า “firmware compile ผ่านไหม” การแยกสองคำถามนี้ช่วยให้รายงานตรงและไม่ปน claim

บทที่ 7: เพิ่ม pack ใน WASM web picker

ฝั่ง web preview เป็นด้านที่ทีมเห็นได้เร็วที่สุด

จุดต่อคือ PACKS ใน:

lab/gif-wasm/web/index.html

เพิ่ม entry:

```
"digimal": {
  dir: "digimal/",
  states: ["idle", "busy", "attention", "celebrate", "dizzy", "sleep", "heart"]
}
```

หลังจากนั้น web app จะอ่าน GIF จาก:

web/gifs/digimal/<state>.gif

ตรงนี้อาจสำคัญมาก เพราะ preview ที่ดีไม่ควรเป็น ธรรมดาที่ห้ล่อกตาได้ แต่ควรใช้ decoder path เดียวกับที่ระบบตั้งใจทดสอบ ใน source นี้คือ gif-wasm ที่ compile decoder เป็น WebAssembly แล้ว expose API ให้ JavaScript เรียก

หลักฐานที่จับได้คือ screenshot:

/tmp/digimal-wasm-preview.png

ภาพนี้มีค่าเพราะมันไม่ใช่แค่ thumbnail ของไฟล์ GIF แต่เป็น browser rendering หลังจาก pack ถูกเลือกใน UI แล้ว

สำหรับ Discord หรือทีม review ต้องส่งภาพหรือ zip เป็น attachment จริง ไม่ส่ง local path เนย ๆ เพราะ path /home/axezi/... เปิดได้เฉพาะเครื่องนี้ คนอื่นดูไม่ได้

บทที่ 8: WASM decode proof

ภาพ preview สวยยังไม่พอ ต้องถาม decoder ว่าอ่านได้จริงไหม

ในงานนี้ใช้ Node script โหลด web/gifdec.js แล้วเรียก WASM decoder กับทุก state ของ digimal ผ่าน fetch shim ที่อ่านไฟล์ local ได้

ผลที่ต้องพิสูจน์มีสามอย่าง:

```
rc == 0
width == 96
height == 100
frames >= 1
```

คำสั่งตรวจมีรูปแบบนี้:

```
cd lab/gif-wasm
node - <<'NODE'
const fs = require('fs');
const path = require('path');
global.fetch = async (url) => {
  const p = String(url).replace(/^file:\/\/\/, '');
  const data = fs.readFileSync(p);
  return {
    ok: true,
    arrayBuffer: async () =>
      data.buffer.slice(data.byteOffset, data.byteOffset + data.byteLength)
  };
};
const makeModule = require('./web/gifdec.js');
const states = ['idle', 'busy', 'attention', 'celebrate', 'dizzy', 'sleep', 'heart'];
makeModule({ locateFile: (p) => path.resolve('web', p) }).then(M => {
  for (const state of states) {
    const file = path.join('web/gifs/digimal', `${state}.gif`);
    const g = fs.readFileSync(file);
    const p = M._malloc(g.length);
    M.HEAPU8.set(g, p);
    const rc = M._gif_open(p, g.length);
    const w = M._gif_width();
    const h = M._gif_height();
    let frames = 0;
    const dptr = M._malloc(4);
```

```

    for (let i = 0; i < 100; i++) {
      const r = M._gif_play(dp_ptr);
      if (r < 0) break;
      frames++;
      if (r === 0) break;
    }
    M._free(dp_ptr);
    M._gif_close();
    M._free(p);
    if (rc !== 0 || w !== 96 || h !== 100 || frames < 1) {
      throw new Error(`${state}: rc=${rc} ${w}x${h} frames=${frames}`);
    }
    console.log(`${state}: rc=${rc} ${w}x${h} frames=${frames}`);
  }
});
NODE

```

นี่เป็น proof ที่ดีกว่า “เปิดแล้วเหมือนจะเห็น” เพราะมันใช้ decoder จริงและ fail ได้ถ้า GIF ไม่ถูก format

บทที่ 9: LittleFS proof

Firmware asset ไม่จบที่ folder ใน repo มันต้องเข้า filesystem image ได้ด้วย

ใน desk-pet pipeline, character GIFs อยู่ใน LittleFS แล้ว firmware อ่านจาก path เช่น:

/characters/digimal/idle.gif

ดังนั้น LittleFS proof คือการยืนยันว่า tree ใต้ data/ ถูก pack ได้ใน image ขนาดที่ board ใช้จริง โดยไม่ต้องอ้างว่า app binary compile แล้ว

proof image ที่สร้างไว้คือ:

/tmp/digimal-storage-proof.bin

ขนาด proof:

3145728 bytes

ตัวเลขนี้สำคัญเพราะตรงกับ shared 3 MB LittleFS ที่ workshop พูดถึง ถ้า asset pack ใหญ่เกิน หรือ path สร้างไม่ได้ ขั้นนี้จะ fail ก่อนถึง hardware

LittleFS proof เป็นการลดความเสี่ยงแบบถูกจุด มันไม่แทน hardware flash แต่ช่วยแยกว่า pack ไม่ได้พังตั้งแต่ storage layer

บทที่ 10: Compile boundary

รายงานที่ดีต้องบอกทั้งสิ่งที่ผ่านและสิ่งที่ยังไม่ผ่าน

ในเครื่องนี้ asset validation ผ่าน แต่ firmware compile ยังติด environment:

missing ~/esp/esp-idf

missing python3.13

นี่เป็นคนละปัญหากับ GIF schema ถ้า report รวมกันว่า “compile failed” เฉย ๆ คนอ่านอาจคิดว่า pack ผิด แต่หลักฐานตอนนั้นบอกว่า pack อ่านได้, manifest อยู่ครบ, WASM decode ได้, browser preview ได้ และ LittleFS image สร้างได้

claim ที่ถูกจึงเป็น:

digimal character pack is valid against the local asset contract and web decoder.
Full firmware build is blocked by the local ESP-IDF/Python toolchain.

การพูดแบบนี้ไม่ใช่ข้อแก้ตัว แต่เป็นการแบ่ง boundary ให้ reviewer ตรวจสอบได้ ถ้าเครื่องของทีมมี ESP-IDF และ Python version ที่ตรง เขาสามารถเอา pack นี้ไป build ต่อได้โดยไม่ต้องเดาว่าปัญหาอยู่ตรงไหน

ในงาน embedded, environment เป็นส่วนหนึ่งของระบบเสมอ แต่ไม่ควรให้ environment blocker ลบหลักฐานที่ผ่านแล้ว

บทที่ 11: ส่ง proof แบบทีมดูได้

กุญสำคัญของงานใน Discord คืออย่าส่ง local path เป็น deliverable

Path แบบนี้มีประโยชน์ในเครื่องเรา:

```
/home/axezi/atom/.cc-connect/work/...  
/tmp/digimal-wasm-preview.png
```

แต่เพื่อนในทีมเปิดไม่ได้ สิ่งที่ต้องส่งคือไฟล์จริงหรือ URL ที่เข้าถึงได้

สำหรับงานนี้ proof ที่ส่งควรมี:

```
digimal-proof.zip  
  characters/digimal/*.gif  
  characters/digimal/manifest.json  
  web/gifs/digimal/*.gif  
  digimal-wasm-preview.png  
  README.txt
```

คำสั่งส่งผ่าน bridge:

```
cc-connect send --file /tmp/digimal-proof.zip --message "digimal desk-pet proof"
```

ถ้าเป็นภาพ ต้องส่งภาพจริง ไม่ใช่บอกว่า "ดูที่ path นี้" เพราะผู้ตรวจอยู่คนละ context กับ shell ของเรา

lesson ของบทนี้ตรงกับ feedback ที่เคยได้: เวลาที่ให้ส่งให้ส่งรูป ไฟล์ หรือวิดีโอ ต้องส่งจริง ไม่ใช่ส่ง path ให้คนอื่นเปิดไม่ได้

บทที่ 12: Workshop hygiene

งาน public workshop ต้องมีวินัยกว่างาน local demo

สิ่งที่ควรทำ:

- อ่าน repo และ source ก่อนเลือก implementation lane
- ใช้ running number และ folder แยกเมื่อต้อง submit
- ไม่ส่ง firmware binary ที่ไม่รู้ header ถูกหรือไม่
- ไม่ re-add asset หรือ bin ที่ reviewer เอาออกเพราะเสี่ยง
- ส่ง proof ที่คนอื่นเปิดได้จริง
- บอก compile/build status ตามหลักฐาน ไม่ตามความหวัง

digimal เป็นตัวอย่างของแนวทางที่เล็กแต่ตรวจได้:

```
source found  
pack contract matched
```

GIF dimensions validated
WASM decoder passed
browser preview captured
LittleFS image created
firmware compile blocker named
proof zip attached

ถ้าจะไปต่อ ขึ้นถัดไปไม่ใช่เขียนระบบใหม่ แต่คือเตรียม environment ให้ตรง:

ESP-IDF path
Python version required by Makefile
board target and display config
hardware flash + serial capture

หลังจากนั้นจึงค่อย claim ได้ว่า firmware body เล่น digital บนจอจริง

หนังสือเล่มนี้จบที่จุดที่ honest engineering ควรจบ: เรามี pack ที่ผ่าน asset/runtime proof แล้ว และมีรายการ
ชัดเจนของสิ่งที่ต้องใช้เพื่อพิสูจน์ hardware ต่อ